

Building the Best Church Website in Jamaica: Supabase-First Product, Architecture, Commerce, and Delivery Plan

Executive summary

The recommended product is **not merely a church website**. It should be designed as a unified **digital church platform** for New Testament Church of God, Bull Bay: a public-facing website, member portal, pastoral workspace, church-management system, media library, giving platform, e-commerce store, communications hub, reporting system, and future multi-campus foundation.

The strongest architecture for the requirements is:

Next.js 16 + React + TypeScript → Supabase Auth/PostgreSQL/Storage/RLS → Jamaica-capable payment providers → dedicated media streaming → communications/integrations.

Next.js 16.3 is current as of August 2026, and Supabase officially supports Next.js SSR through `@supabase/ssr`, with sessions stored in cookies. ¹ Supabase is particularly well suited because authentication integrates directly with PostgreSQL Row Level Security, allowing authorization to be enforced in the database rather than trusting the browser. ²

The system should have four major experiences:

Experience	Goal
Public website	Make the church easy to discover, understand, visit, watch, contact, support and buy from
Member portal	Give members one place for profile, household, giving, groups, events, volunteering, orders and notifications
Pastor/clergy workspace	Make sermons, pastoral care, ministry insight, service planning and communication efficient without exposing unrelated financial/admin data
Admin/staff workspace	Run people, events, groups, volunteers, media, giving, shop, communications, content, campuses, permissions, reporting and integrations

This scope is consistent with capabilities established church-management platforms have converged on: Planning Center, for example, separates people management, groups, check-ins, volunteer/service scheduling and member-facing functions; Pushpay combines church management, communications, groups, scheduling and giving; Subsplash combines sites, apps, giving and livestreaming. ³ The

opportunity here is to combine those concepts in a **church-owned Supabase data model** rather than becoming dependent on a single SaaS church-management vendor.

For Jamaica specifically, the most important early architectural decision is payments. **Do not make Stripe the default payment gateway for a Jamaican church entity.** Stripe's current Payments global-availability page does not list Jamaica as a supported merchant country. Stripe does offer Global Payouts to Jamaica, but payouts are a different product from opening a Jamaican Stripe Payments merchant account. ⁴ PayPal provides Jamaica-specific business checkout, invoices and recurring payment capabilities, while WiPay markets Caribbean card acceptance/local settlement and First Atlantic Commerce/Powertranz specifically serves Caribbean and Central American merchants. ⁵

My recommendation is therefore:

Primary architecture

- Next.js 16 App Router, TypeScript and React.
- Supabase Pro initially.
- Vercel Pro for the web application.
- PayPal plus either WiPay or Powertranz after merchant underwriting/commercial comparison.
- Cloudflare Stream for owned/private/premium video and YouTube for public reach.
- SendGrid for transactional and bulk church email.
- Twilio only for high-value SMS alerts because Jamaica SMS pricing is comparatively expensive.
- Google Calendar synchronization plus downloadable `.ics` feeds.
- GA4/Search Console for public-site/e-commerce analytics.
- QuickBooks Online or Xero export/API as accounting requirements mature.
- Webhook-based Zapier/Make automation.
- External backup of Supabase Storage in addition to database backups.

Supabase Pro currently begins at **US\$25/month**, including 100,000 MAU, 8 GB database disk, 100 GB file storage, 250 GB egress and seven-day daily database backups; paid plans include US\$10 of compute credit.

⁶ Vercel Pro currently starts at **US\$20/month** with CI/CD, CDN, WAF/DDoS functionality and included usage allowances. ⁷ That makes an excellent production foundation without requiring a large infrastructure team.

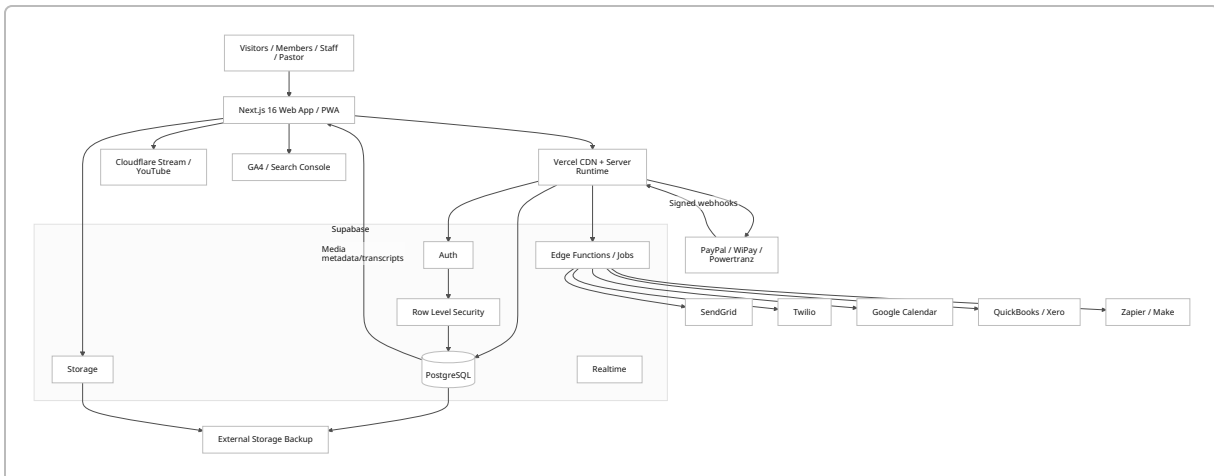
A starter implementation package has also been created for this project:

[Download the Bull Bay Church Supabase/Next.js starter ZIP](#)

It contains 31 files including a Next.js starter, public pages, member/admin shells, a sample cart, payment webhook scaffolding, Supabase clients, an extensive PostgreSQL migration with RLS, development seed data, CI configuration, deployment script, security notes and architecture documentation.

The screenshot supplied for the project identifies the intended church as **New Testament Church of God, Bull Bay**, so the starter uses that as branding context. Service times, leadership names, legal entity details and other operational information remain placeholders until church staff verify them.

The target architecture is:



The strategic principle is simple: **Supabase should own the canonical church data, while external vendors perform specialized functions such as card processing, video delivery, email delivery and accounting synchronization.**

Product scope and role-based capability map

Priorities below use:

P0 = required for initial public launch or secure operation.

P1 = required for a complete first church-management release.

P2 = enhancement after the operating model has proved itself.

The APIs shown are recommended application contracts. Public, RLS-safe reads can sometimes be performed directly through Supabase; privileged operations, payments, exports, refunds, integrations and confidential actions should go through server-side Next.js routes.

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P0	Public home	Home, service-time card, hero, ministries, upcoming events, latest sermon, giving CTA, shop CTA, map/directions	<code>GET /api/public/home</code>	hero, service times, campus, address, featured events, sermons, CTAs	Anonymous read of published data
P0	About / beliefs / leadership	About, beliefs, history, leadership, ministries	<code>GET /api/pages/:slug</code>	title, slug, body, image, SEO metadata, status	Public published read; content staff write

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P0	Locations / multi-site public view	Locations directory, campus detail, service times, directions	GET /api/campuses , GET /api/campuses/:slug	name, address, coordinates, phone, email, service schedule	Public read of active campuses
P0	Events	Event listing, filters, event detail, RSVP, confirmation	GET /api/events , GET /api/events/:slug , POST /api/events/:id/register	title, description, campus, start/end, location, image, capacity, registration rules	Public read; users manage own RSVP; event staff manage
P0	Calendar	Month/list calendar, ministry filters, campus filters, Add-to-Calendar	GET /api/calendar , GET /api/calendar.ics , GET /api/calendar/:id.ics	event dates, timezone, location, recurrence, category	Public events visible; private events role-scoped
P0	Sermons	Sermon library, sermon detail/ player, series, speakers, topics	GET /api/sermons , GET /api/sermons/:slug	title, speaker, Scripture, series, topics, summary, video/audio IDs, thumbnail, duration	Anonymous read published sermons; media editors write
P0	Media search	Search screen, transcript results, topic/ Scripture filters	GET /api/search?q= , later POST /api/search/semantic	transcript, search vector, topics, Scripture, date, speaker	Search limited to records user may read
P0	Livestream	Watch Live, countdown, scheduled stream, prayer/ contact CTA	GET /api/live	provider ID, status, scheduled start, title, chat link	Public where stream is public

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P0	Giving	Give, choose fund, one-time/ recurring, checkout, confirmation	POST /api/giving/checkout , provider webhook	donor, amount, currency, fund allocation, provider IDs, status	Donor reads own gifts; finance role reads all
P0	Shop/catalog	Shop, category, search, product detail	GET /api/shop/products , GET /api/shop/products/:slug	SKU, type, name, description, images, price, currency, tax class, inventory	Public read active products; shop staff manage
P0	Cart/checkout	Cart, address, delivery, payment, order confirmation	POST /api/cart , POST /api/shop/checkout	lines, quantities, price snapshot, tax, shipping, customer, totals	Customer owns cart/ order; server determines prices
P0	Digital product delivery	My Downloads, licence view, download history	GET /api/me/downloads , POST /api/downloads/:id/url	entitlement, asset path, expiry, max downloads, licence	Purchaser only; shop support can revoke/ reissue
P0	Contact / first visit	Contact, Plan Your Visit, newcomer form	POST /api/contact , POST /api/visitors	name, email, phone, interests, visit date, consent	Public submit; designated follow-up staff read
P0	Prayer requests	Prayer form, confidentiality selector, member prayer history	POST /api/prayer , GET /api/me/prayer	submitter, text, privacy level, assignment, state	Strict care-team access; submitter sees own where appropriate
P1	Small groups public/member	Group directory, detail, join/ request, My Groups	GET /api/groups , POST /api/groups/:id/join , GET /api/me/groups	name, leader, category, schedule, location area, capacity, membership	Public group metadata; membership details role-controlled

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P1	Volunteer opportunities	Serve page, opportunity detail, availability, My Schedule	GET /api/volunteer , POST /api/volunteer/:id/apply , GET /api/me/serving	ministry, role, skills, shift, status, availability	Member owns availability; ministry leaders schedule
P1	Member profile	Account, personal info, profile photo, consent, privacy	GET/PATCH /api/me	names, phone, email, DOB, avatar, status, communication consent	User self-service; authorized people staff update
P1	Household/ family profile	Household page, spouse/ child links, emergency contact	GET/PATCH /api/me/household	household, relationship, contact, guardians	Household access with safeguards; children restricted
P1	Member giving history	Gifts, funds, recurring gifts, receipts/ statements	GET /api/me/giving , GET /api/me/receipts/:id	amount, fund, date, provider, receipt	Donor only plus finance roles
P1	Member orders	Orders, invoice, refund state, delivery tracking	GET /api/me/orders , GET /api/me/orders/:id	line items, totals, delivery, payment, refunds	Purchaser only plus shop/ finance support
P1	Notification preferences	Communication settings, subscriptions	GET/PATCH /api/me/notifications	email, SMS, push, event/ group/ministry preferences	User controls own preferences
P1	Admin dashboard	KPI cards, tasks, approvals, recent activity	GET /api/admin/dashboard	upcoming events, visitors, giving summaries, orders, volunteer gaps	Staff sees only metrics allowed by permissions

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P1	People CRM	People list, profile, household, tags, journey/history	<code>/api/admin/people/*</code>	membership status, contact data, household, tags, milestones, campus	<code>people.read</code> / <code>people.write</code> ; sensitive fields separately scoped
P1	Visitor follow-up	Visitor inbox, pipeline, tasks, notes	<code>/api/admin/visitors/*</code>	first visit, interests, assigned owner, next action, status	Assimilation team only
P1	Event administration	Event editor, registrations, attendance, export	<code>/api/admin/events/*</code>	public fields + registration schema, capacity, attendance	<code>events.manage</code>
P1	Check-in / attendance	Check-in kiosk, attendance dashboard, guardian flow	<code>/api/admin/checkins/*</code>	person, event, timestamp, check-in/out, pickup token	Check-in team; safeguarding restrictions
P1	Group administration	Group editor, leaders, members, attendance	<code>/api/admin/groups/*</code>	group, leaders, roster, attendance	<code>groups.manage</code> ; leaders restricted to their groups
P1	Volunteer administration	Teams, positions, schedules, conflicts, reminders	<code>/api/admin/volunteers/*</code>	opportunities, shifts, assignment, availability	<code>volunteers.manage</code> ; leaders own ministry
P1	Content/CMS	Pages, navigation, announcements, posts, drafts/preview	<code>/api/admin/content/*</code>	slug, blocks, status, author, publish date, SEO	<code>content.manage</code>
P1	Media administration	Upload/import, transcript editor, sermon metadata, thumbnails	<code>/api/admin/media/*</code> , <code>/api/admin/sermons/*</code>	media IDs, files, transcript, metadata, provider	<code>sermons.manage</code>

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P1	Giving administration	Donation ledger, funds, reconciliation, receipts	<code>/api/admin/giving/*</code>	donor, amount, provider transaction, fund splits, status	Separate <code>giving.read</code> / <code>giving.manage</code>
P1	Shop administration	Products, variants, orders, refunds, licences, inventory	<code>/api/admin/shop/*</code>	product, SKU, price, stock ledger, order, entitlement	<code>shop.manage</code> ; refund power optionally separate
P1	Communications	Audience builder, composer, preview, scheduled campaigns	<code>/api/admin/campaigns/*</code>	recipients/segment, channel, template, state, metrics	<code>communications.send</code> ; consent filters mandatory
P1	Reporting	Attendance, growth, giving, shop, engagement, volunteers	<code>/api/admin/reports/*</code>	aggregated metrics, date/campus filters	<code>reports.read</code> ; financial reports require finance permission
P1	Roles & permissions	Roles, permission matrix, staff invitations, audit	<code>/api/admin/roles/*</code>	role, permission, campus scope, grantor, timestamps	<code>roles.manage</code> ; ideally few super-admins
P1	Audit trail	Security/activity log	<code>GET /api/admin/audit</code>	actor, action, entity, metadata, IP/session metadata where appropriate	Security/super-admin role
P1	Multi-site administration	Campus editor, campus switcher, campus dashboards	<code>/api/admin/sites/*</code>	campus, address, contacts, service schedule, settings	<code>sites.manage</code> ; other roles campus-scoped

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P1	Pastor dashboard	Pastoral dashboard, care tasks, sermon schedule, ministry trends	GET /api/pastor/dashboard	upcoming preaching, care assignments, ministry summaries	Clergy access; no automatic shop/finance access
P1	Sermon planning	Series planner, sermon editor, Scripture/topics, publish workflow	/api/pastor/sermons/*	series, title, text, attachments, media, transcript	Pastor and designated media team
P1	Pastoral care	Care case list/detail, confidential notes, follow-ups	/api/pastor/care/*	subject, category, notes, owner, state, access list	Explicit care-team access; access audited
P1	Service planning	Service plan, order of service, volunteers, assets	/api/pastor/services/*	service date, elements, songs/media, notes, assigned people	Worship/clergy team
P1	Pastoral communications	Message congregation/groups/leaders	/api/pastor/messages/*	audience, message, channel, send state	Approved audiences only
P2	Resource booking	Rooms/assets calendar and reservations	/api/admin/resources/*	room/equipment, availability, reservation	Ministry leaders request; staff approve
P2	Workflows/forms	Form builder, automation rules, task queues	/api/admin/forms/*, /api/admin/workflows/*	field schema, response, trigger, action	Form owner / workflow admins
P2	Semantic sermon/Bible-topic search	Natural-language search and related sermons	/api/search/semantic	transcript chunks, embeddings, permissions	Same RLS as source content

Priority	Capability / role	Required UI screens	Recommended API contract	Core data	Permission model
P2	PWA/push	Install prompt, notification center	/api/push/*	device/subscription token, preferences	Member opt-in
P2	Software licence management	Licence dashboard, activations, revoke/reset	/api/me/licenses, /api/admin/licenses/*	licence hash, product, entitlement, activations, expiry	Buyer/support only
P2	Accounting automation	Sync dashboard, failures/retry	/api/admin/integrations/accounting/*	external IDs, sync status, journal/invoice mapping	Finance admins
P2	Workflow automation	Integration/webhook management	/api/admin/integrations/webhooks/*	event, endpoint, secret, retries	Integration administrators

Several mature church-management systems specifically treat people, groups, child/family check-in, volunteer scheduling, service planning and registrations as distinct operational domains, which validates keeping them modular rather than trying to force everything into a generic “members” table. ⁸

A crucial permission decision: “Admin” should **not** mean “can see everything.” Use granular capabilities such as:

```

people.read
people.write
events.manage
groups.manage
volunteers.manage
sermons.manage
content.manage
communications.send
giving.read
giving.manage
shop.manage
refunds.approve
reports.read
care.read
care.manage
roles.manage

```

```
sites.manage
integrations.manage
```

A media administrator, for example, should be able to publish sermons without seeing donation histories. A shop manager should manage orders without reading counseling notes. A pastor should see assigned pastoral-care cases without automatically getting access to raw e-commerce or individual giving data.

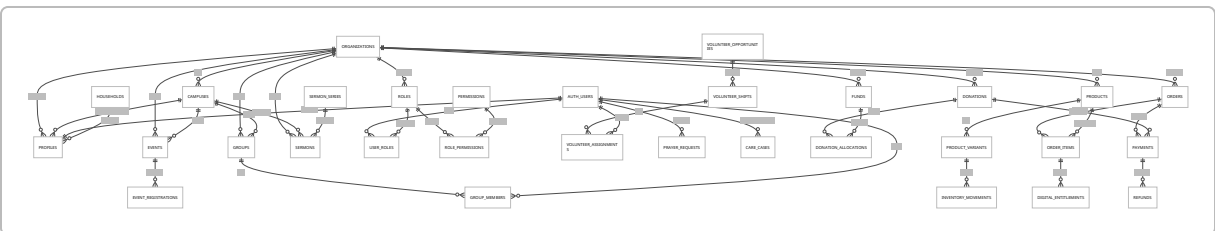
That separation is especially important because the church will hold materially sensitive personal information.

Supabase architecture, schema, APIs, and authentication

Supabase Auth supports passwords, magic links, OTP, social login and SSO and integrates JWT-authenticated requests directly with PostgreSQL RLS. ⁹ For Next.js SSR, Supabase's recommended tooling uses `@supabase/ssr` and cookie-backed sessions. ¹⁰

The database should be deliberately **multi-campus from day one**, even though Bull Bay starts as one site. This costs little architecturally and avoids painful retrofitting later.

Recommended identity model



The production data dictionary should look approximately like this. The starter ZIP contains the corresponding SQL for the core set.

Table	Important columns and PostgreSQL types	Relationships	Important indexes
organizations	id uuid, name text, slug citext, default_currency char(3), timezone text	Tenant root	unique slug

Table	Important columns and PostgreSQL types	Relationships	Important indexes
campuses	id uuid, organization_id uuid, name text, address fields, latitude numeric, longitude numeric, contact fields	organization	(organization_id, slug)
households	id uuid, organization_id uuid, name text	profiles	organization_id
profiles	id uuid FK auth.users, org/campus/household UUIDs, names, phone, DOB, status, consent booleans	auth user, campus, household	org, campus, household, (org, last_name, first_name)
roles	id uuid, org, code text, name text	user_roles	(org, code) unique
permissions	code text, description text	role_permissions	PK code
role_permissions	role UUID, permission code	roles ↔ permissions	compound PK
user_roles	org, user, role, optional campus, grantor	auth, role, campus	(org, user)
audit_logs	actor, action, entity type/id, metadata jsonb, timestamp	org/user	(org, created_at desc)
events	org/campus, title/slug, description, start/end timestamptz, capacity, visibility/status	campus	(org, status, starts_at)

Table	Important columns and PostgreSQL types	Relationships	Important indexes
event_registrations	event, user/guest, quantity, status	event/user	event, user
groups	org/campus, name, category, meeting pattern, capacity, visibility	campus	(org, is_active)
group_members	group, user, role, status	group/user	user
volunteer_opportunities	org/campus, title, ministry, active	campus	org/campus
volunteer_shifts	opportunity, event, start/end, slots	opportunity/event	starts_at
volunteer_assignments	shift, user, status	shift/user	compound PK
sermon_series	org, title, slug, description, image	sermons	(org, slug)
sermons	org/campus, series, title, speaker, Scripture array, topics array, transcript, media IDs, publish state	series/campus	published date + GIN text search
prayer_requests	org, submitter, body, visibility, status, assignee	user	(org, status, created_at)
care_cases	org, subject, owner, summary, confidential notes, status	users	(org, owner, status)
funds	org, code, name, active	donations	(org, code)
donations	donor, amount bigint minor units, currency, provider IDs, payment status, receipt	fund allocation/payment	provider payment unique; donor/date
donation_allocations	donation, fund, amount_minor	donation/fund	compound PK

Table	Important columns and PostgreSQL types	Relationships	Important indexes
<code>products</code>	org, name, slug, kind enum, state, price_minor, taxable, image paths, digital path	variants/order items	<code>(org,status,kind)</code>
<code>product_variants</code>	product, SKU, name, override price, track inventory	product	unique SKU
<code>inventory_movements</code>	variant, quantity delta, reason, reference	variant	<code>(variant,created_at)</code>
<code>orders</code>	org, user, order number, status, totals, customer, addresses <code>jsonb</code>	user/payment/items	user/date; org/status/date
<code>order_items</code>	order, product/variant, snapshot name/SKU/ price, qty/tax/total	order/product	order
<code>payments</code>	org, order OR donation, provider, external ID, amount, status, payload	order/donation	provider external ID unique
<code>refunds</code>	payment, provider refund ID, amount, reason, status	payment	payment
<code>digital_entitlements</code>	user, order item, email, expiry, download limit/count, licence hash	user/order item	user
<code>notification_preferences</code>	user, channel/ category booleans	user	user PK
<code>webhook_events</code>	provider, external event ID, event type, payload, processed/ error state	payments/ integrations	<code>(provider,event_id)</code> unique

Additional P1/P2 tables should eventually include `pages`, `posts`, `navigation_items`, `forms`, `form_submissions`, `campaigns`, `campaign_recipients`, `group_meetings`, `group_attendance`, `checkins`, `children`, `guardians`, `resources`, `resource_bookings`,

shipping_zones, shipping_methods, subscriptions, license_activations, integration_connections, sync_jobs, sermon_transcript_chunks and embedding_jobs.

Money should always use integer minor units, for example JMD \$3,500.00 as 350000, rather than floating point arithmetic.

Order prices must be snapshotted. If a product costs JMD 3,500 today and changes to JMD 4,000 next month, historical invoice lines must continue to show the exact price that was charged.

Inventory should use a ledger rather than only a mutable stock_count. The balance becomes the sum of movements:

```
+24 initial stock
-1 customer order
-1 customer order
+1 returned item
-2 manual adjustment
-----
21 available
```

This creates a much stronger audit trail.

Supabase recommends enabling RLS for tables exposed through its APIs; policies then act as additional row-level predicates. Supabase also warns against using raw_user_meta_data for authorization because the user can modify it; raw_app_meta_data is safer, although for this church I would keep canonical authorization in PostgreSQL role tables. ¹¹

A representative RLS design is:

```
create table public.user_roles (
  organization_id uuid not null
    references public.organizations(id) on delete cascade,

  user_id uuid not null
    references auth.users(id) on delete cascade,

  role_id uuid not null
    references public.roles(id) on delete cascade,

  campus_id uuid
    references public.campuses(id) on delete cascade,

  primary key (user_id, role_id, campus_id)
);
```

```

create or replace function public.has_permission(
    org uuid,
    permission text
)
returns boolean
language sql
stable
security definer
set search_path = ''
as $$
    select exists (
        select 1
        from public.user_roles ur
        join public.role_permissions rp
        on rp.role_id = ur.role_id
        where ur.user_id = (select auth.uid())
        and ur.organization_id = org
        and rp.permission_code = permission
    );
$$;

alter table public.events enable row level security;
alter table public.profiles enable row level security;
alter table public.donations enable row level security;
alter table public.care_cases enable row level security;

create policy "public can read published events"
on public.events
for select
to anon, authenticated
using (
    status = 'published'
    and visibility = 'public'
);

create policy "member can read own profile"
on public.profiles
for select
to authenticated
using (id = (select auth.uid()));

create policy "people staff can read profiles"
on public.profiles
for select
to authenticated
using (
    public.has_permission(
        organization_id,

```

```

        'people.read'
    )
);

create policy "donor sees own gifts"
on public.donations
for select
to authenticated
using (
    donor_user_id = (select auth.uid())
);

create policy "finance staff sees donations"
on public.donations
for select
to authenticated
using (
    public.has_permission(
        organization_id,
        'giving.read'
    )
);

create policy "care team reads care cases"
on public.care_cases
for select
to authenticated
using (
    owner_user_id = (select auth.uid())
    or public.has_permission(
        organization_id,
        'care.read'
    )
);

```

Never expose a Supabase `service_role` /secret key in browser JavaScript because service-level access can bypass RLS. ¹¹

Storage should follow the same principle. Supabase Storage integrates authorization with PostgreSQL RLS; private buckets can be served through signed URLs. ¹²

Recommended buckets:

Bucket	Public?	Purpose
<code>public-site</code>	Yes	Hero images, intentionally public page assets

Bucket	Public?	Purpose
<code>sermon-audio</code>	Usually yes	Public sermon audio
<code>member-avatars</code>	No	Member profile images
<code>digital-products</code>	No	Paid files/software
<code>pastoral-attachments</code>	No	Sensitive care documentation
<code>receipts</code>	No	Giving/order documents
<code>exports</code>	No	Short-lived CSV/PDF exports

Paid digital goods should **never** be placed in a public bucket. Generate a short-lived signed URL only after checking an entitlement. Supabase private buckets and signed URLs are specifically designed for this pattern. ¹³

Recommended authentication journey

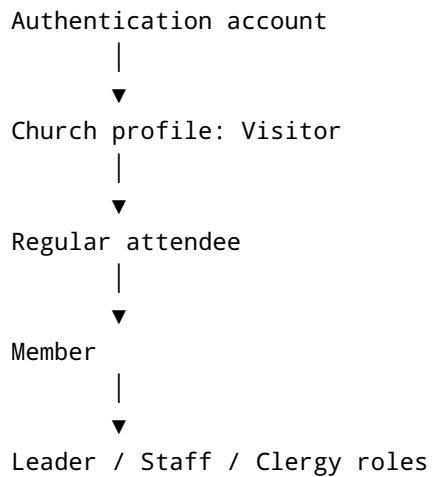
Audience	Authentication	Provisioning
Visitor browsing website	None	Anonymous
New visitor submitting form	None + CAPTCHA/rate limit	CRM visitor record only
Member	Email magic link/OTP initially; optional password; Google login	Create profile after verified identity
Staff	Invitation-only	Role manually granted
Pastor/clergy	Invitation-only	Clergy role + explicit care permissions
Super admin	Invitation-only	Minimal number of accounts
Large future organization	SSO via supported enterprise identity provider	Organization-managed

Supabase supports email/password, magic link, OTP, social login and SSO. ⁹

For members, Google login plus email magic link is a low-friction default. For staff and clergy, require MFA. Supabase Auth supports TOTP and phone-based MFA and exposes assurance levels that applications can use when guarding sensitive operations. ¹⁴

For example, an ordinary authenticated member session could operate at `aa11`, while refunds, giving exports, permissions management and pastoral notes could require `aa12`.

Do not make account creation equal church membership. Use two concepts:



That preserves the distinction between “someone created an account” and “the church recognizes this person as a member.”

For member onboarding:

```
GET /login
POST Supabase Auth OTP/OAuth
GET /auth/callback
POST /api/onboarding/profile
POST /api/onboarding/household
POST /api/onboarding/preferences
GET /member
```

For staff onboarding:

```
POST /api/admin/invitations
→ invitation email
→ verified Supabase account
→ force MFA enrollment
→ role/campus assignment
→ audit log
```

The SQL already included in the downloadable starter implements the foundation for this model.

Commerce, media, and integrations

The church shop should be treated as a serious e-commerce system rather than a page containing payment buttons.

Products should support:

physical
digital
software
service
subscription

Examples include church shirts, books, ministry materials, sermon/study downloads, digital courses, software the church creates, event merchandise, subscriptions or licence-based products.

Recommended payment strategy for Jamaica

Provider	Jamaican church fit	Cards / checkout	Recurring	Local relevance	Recommendation
PayPal	Strong starting option	PayPal says its Jamaica business checkout supports online payment methods and card checkout	Yes; Jamaica checkout advertises subscription support	Jamaica-specific merchant/business pages	Start here unless underwriting/accounting says otherwise. ¹⁵
WiPay	Strong regional candidate	Markets card acceptance, payment links and QR payments	Confirm exact Jamaica product terms during onboarding	Built for Caribbean markets and advertises local-bank settlement	Run commercial/technical evaluation alongside PayPal. ¹⁶
Powertranz / First Atlantic Commerce	Strong bank/acquirer-oriented option	Online debit/credit-card gateway	Confirm merchant agreement	Designed for Central America/Caribbean and integrated with regional banks	Strong option for a more institutional setup. ¹⁷

Provider	Jamaican church fit	Cards / checkout	Recurring	Local relevance	Recommendation
Stripe Payments	Poor default for Jamaican entity today	Excellent developer tooling where merchant country is supported	Yes in supported regions	Jamaica is not on Stripe's current direct Payments merchant availability list	Do not architect around it unless the church has an eligible supported-country entity. ¹⁸

Do not confuse Stripe Global Payouts—which can deliver payouts to Jamaican recipients—with a Jamaican merchant being able to open standard Stripe Payments. ¹⁹

The application should implement a payment-provider adapter:

```
interface PaymentGateway {
  createCheckout(input: CheckoutInput): Promise<CheckoutSession>;
  verifyWebhook(headers: Headers, rawBody: string): Promise<PaymentEvent>;
  refund(input: RefundInput): Promise<RefundResult>;
  getPayment(externalId: string): Promise<PaymentStatus>;
}
```

Then:

```
PAYMENT_PROVIDER=paypal

or

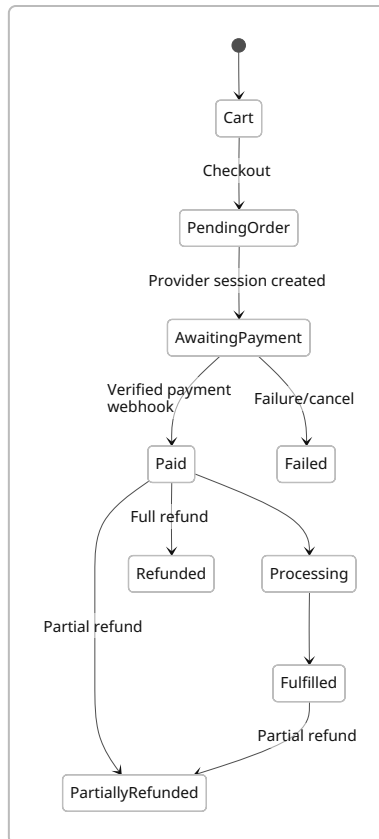
PAYMENT_PROVIDER=wipay

or

PAYMENT_PROVIDER=powertranz
```

This prevents the church's entire commerce system becoming coupled to one processor.

The payment state machine must be webhook-first:



Never mark an order paid simply because a user's browser returns to `/checkout/success`. The trusted event is the authenticated provider webhook.

Provider event IDs should have a unique constraint:

```

create table webhook_events (
  id bigint generated always as identity primary key,
  provider text not null,
  provider_event_id text not null,
  event_type text not null,
  payload jsonb not null,
  processed_at timestampz,
  unique(provider, provider_event_id)
);

```

That makes webhook processing idempotent.

For payment-page security, PCI DSS applies to entities that store, process, transmit, or can affect payment-account data, and PCI SSC has tightened expectations around e-commerce payment-page scripts and tampering. ²⁰ The simplest church architecture is therefore to minimize exposure to cardholder data through provider-hosted checkout/components and never store raw PAN or CVV. PCI SSC notes that SAQ eligibility depends heavily on where payment-page elements originate, so the church should confirm its

exact compliance scope with its acquiring provider rather than assuming a hosted payment automatically eliminates all PCI obligations. ²¹

Digital products and church-created software

After verified payment:

```
Payment webhook
  ↓
order = paid
  ↓
create digital_entitlement
  ↓
email buyer
  ↓
buyer opens My Downloads
  ↓
server verifies entitlement
  ↓
Supabase generates 5-minute signed URL
  ↓
download logged
```

An entitlement could contain:

```
id uuid
user_id uuid
order_item_id uuid
email citext
expires_at timestamptz
max_downloads integer
download_count integer
licence_key_hash text
revoked_at timestamptz
created_at timestamptz
```

For software products, add:

```
software_licenses
license_activations
release_versions
download_events
```

Store **hashes** of licence secrets where possible, not reusable plaintext secrets.

Physical inventory

Use:

- product → variants,
- SKU per variant,
- inventory-movement ledger,
- reservation before redirecting to payment,
- reservation expiration,
- restock rules for cancellations/refunds,
- low-stock notification.

Fulfilment choices should include:

```
Pickup at Bull Bay
Kingston / St. Andrew delivery
Other Jamaica parish delivery
International shipping
Digital delivery
```

Avoid putting shipping logic directly into product code. Model shipping zones and methods so rates can change without redeploying the site.

Tax architecture

Jamaica's standard General Consumption Tax rate is 15%, according to current Jamaican government guidance, while individual goods/services may be zero-rated, exempt or otherwise treated differently. ²²

The application should therefore model:

```
taxable boolean
tax_class
tax_rate_snapshot
tax_minor
```

but **do not hard-code every church product to 15%**. The church's accountant should determine:

- whether the legal church entity is registered for GCT,
- which shop sales are taxable,
- treatment of digital/software sales,
- treatment of imported inventory,
- invoicing requirements,
- donation-receipt treatment,

- treatment of overseas buyers.

Keep **donations and commercial purchases as different accounting domains**. A JMD 5,000 contribution to the General Fund is not the same transaction as buying a JMD 5,000 software licence.

Orders should support:

```
subtotal
discount
GCT/tax
shipping
total
paid
partially_refunded
refunded
```

Invoices should use immutable UUIDs internally and sequential human-friendly numbers externally, for example:

```
BB-2026-000184
```

Refunds should support partial amounts, reason codes, provider IDs, timestamps, actor and inventory-restock decision.

Media architecture

Do not store all church video as ordinary web files and serve it directly from the application server.

A strong two-tier model is:

Content	Platform
Public Sunday livestream / public discovery	YouTube
Public archive where maximum reach matters	YouTube
Church-owned controlled playback	Cloudflare Stream
Members-only training	Cloudflare Stream with signed access
Paid video courses	Cloudflare Stream with signed access
Audio sermons	Supabase Storage/CDN or dedicated audio platform
Thumbnails	Supabase Storage or video-provider thumbnails
Canonical metadata	Supabase PostgreSQL

Content	Platform
Transcripts	Supabase PostgreSQL
Search embeddings	Supabase PostgreSQL + pgvector

Cloudflare Stream supports live and on-demand video, adaptive streaming and signed URLs for restricted content. ²³ Stream also exposes APIs for captions and thumbnails, making it suitable for a church-owned media library. ²⁴

YouTube provides APIs for managing or retrieving caption tracks and media metadata, making it practical to synchronize public videos into the Supabase sermon catalog. ²⁵

Every sermon record should contain:

```

title
slug
speaker
series
preached_at
campus
Scripture references
topics
summary
thumbnail
duration
video provider
video ID
audio file
captions
transcript
study notes
publication status
SEO metadata

```

Search can launch with native PostgreSQL full-text search:

```

create index sermons_search_idx
on sermons
using gin (
  to_tsvector(
    'english',
    coalesce(title, '') || ' ' ||
    coalesce(summary, '') || ' ' ||
    coalesce(transcript, '')
  )
)

```

```
)  
);
```

Later, Supabase/pgvector can support semantic and hybrid search so users can search concepts such as:

“sermons about trusting God during hard times”

even when that exact phrase does not occur. Supabase documents both vector semantic search and hybrid keyword/vector search patterns. ²⁶ Permissions-aware retrieval can also preserve RLS boundaries. ²⁷

Recommended service integrations

Need	Recommended service	Implementation
Transactional email	SendGrid	Confirmation emails, password-independent notifications, receipts
Newsletters	SendGrid Marketing or dedicated campaign platform	Segment from Supabase; respect consent
SMS	Twilio	Urgent alerts/reminders only
Calendar	Google Calendar API + ICS	Public event feed plus authorized staff sync
Analytics	GA4 + Search Console	Public site, events, shop funnel
Video	YouTube + Cloudflare Stream	Public reach + owned/private content
Accounting	QuickBooks Online or Xero	Daily summarized journals/invoices/refunds
Automation	Zapier / Make	Signed webhook events
Social login	Google; optionally Microsoft/Apple	Supabase Auth
Search	PostgreSQL FTS; later pgvector	Sermons/pages/events
Storage	Supabase Storage	Images, documents, private downloads

SendGrid currently advertises a free-trial allowance and paid Email API plans beginning around US\$19.95/month. ²⁸

Twilio's current Jamaica pricing lists outbound SMS from international numbers or alphanumeric sender IDs at approximately **US\$0.3151 per segment**, making SMS materially more expensive than email. ²⁹ That argues for:

- email for ordinary newsletters,
- push notifications for engaged members later,
- SMS for urgent/high-value reminders,
- careful opt-in and segmentation.

Google Calendar's API is RESTful and supports event creation and synchronization; Google also supports push notifications for watched resources, which is useful for two-way synchronization. ³⁰

A practical strategy is:

```
Supabase event = canonical church event
|
├─ public website
├─ downloadable .ics
└─ Google Calendar sync
```

Do not make the Google Calendar itself the authoritative database.

For accounting, QuickBooks Online's APIs use OAuth-based authorization, while Xero exposes an Accounting API secured through OAuth 2.0. ³¹ I would initially export/reconcile daily batches rather than trying to synchronize every internal database field.

Zapier's REST Hook model is appropriate for events such as `new_member`, `new_donation`, `new_order`, `event_registration`, or `prayer_request_assigned`; REST hooks push changes when they happen rather than continuously polling. ³²

Example events:

```
{
  "event": "order.paid",
  "event_id": "evt_01...",
  "occurred_at": "2026-08-20T15:30:00Z",
  "organization_id": "...",
  "data": {
    "order_id": "...",
    "total_minor": 600000,
    "currency": "JMD"
  }
}
```

Every external webhook should have:

- an HMAC/signature or provider-native signature,
- event ID,
- timestamp,
- retry policy,
- idempotency,
- delivery log.

Security, privacy, compliance, hosting, and operating cost

For a church platform, security matters more than it would for an ordinary brochure site because the database may contain family relationships, attendance, prayer requests, pastoral-care notes, donation history and children-related records.

The security target should be **OWASP ASVS-based**, not simply “passes a vulnerability scanner.” OWASP describes ASVS as a standard for testing technical security controls and as secure-development guidance, while the current OWASP Top Ten remains a baseline for critical web application risks. ³³

The minimum production controls are:

Area	Required control
Authorization	RLS on every exposed table
Authentication	Staff MFA; member MFA optional initially
Staff access	Invitation-only role elevation
Permissions	Least-privilege role matrix
Finance	Giving access separated from ordinary admin
Pastoral care	Explicit care-team permissions and audit
Children	Separate safeguarding design; minimum necessary data
Payments	Never store raw card/CVV; provider-hosted payment
Webhooks	Signature validation + idempotency
Forms	CAPTCHA, rate limiting and abuse controls
Uploads	Type/size validation, private-by-default sensitive files
Sessions	Secure HTTP-only cookie SSR design
Secrets	Server environment only
Logging	Audit permission changes, exports, refunds and care access
Dependencies	Automated dependency/security updates
Browser security	CSP, secure headers, minimal third-party scripts
Backups	Database and file storage separately
Recovery	Periodic restore exercises
Privacy	Data inventory, purpose, consent, retention and deletion rules

Supabase itself recommends RLS as the data-access boundary and warns that secret/service-role credentials must never be exposed in browsers. ¹¹ Supabase Auth provides MFA using TOTP and phone-

based methods, and the application can distinguish first-factor from second-factor authenticated sessions.

14

I would require MFA for:

- pastors,
- administrators,
- finance staff,
- care team,
- anyone able to change permissions,
- anyone able to issue refunds,
- anyone able to export member lists.

Jamaica Data Protection Act

This is not optional design polish. The Office of the Information Commissioner states that data controllers processing personal data must register with the Commissioner and identifies obligations including the eight data-protection standards, breach notification and, in specified cases, appointment of a Data Protection Officer. 34

The OIC describes the standards as addressing:

- fairness and lawfulness,
- purpose limitation,
- minimization,
- accuracy,
- security/technical and organizational measures,
- adequacy,
- retention limitation,
- respect for data-subject rights. 34

The OIC also states that breaches affecting personal data must be reported to the Commissioner within **72 hours after becoming aware**, and affected data subjects must be notified as required. 35

The OIC further identifies an annual Data Protection Impact Assessment obligation and DPO triggers in specified circumstances, including certain sensitive or large-scale processing. 34 The church should therefore have Jamaican counsel/privacy expertise confirm which DPO and DPIA provisions apply to its exact processing operations rather than treating compliance as a one-time website privacy-policy exercise.

Recommended privacy data inventory:

Data	Purpose	Suggested access
Visitor name/contact	Follow-up	Assimilation staff
Member profile	Church administration	People team

Data	Purpose	Suggested access
DOB	Birthday/ministry eligibility where necessary	Restricted people staff
Household relationships	Family ministry	Restricted
Children/guardians	Check-in/safeguarding	Highly restricted
Giving history	Receipts/accounting	Donor + finance
Prayer requests	Prayer/care	Requester + assigned ministry
Counseling/care notes	Pastoral care	Named care team only
Group membership	Discipleship	Member/group leader/staff
Purchase history	Fulfilment/accounting	Customer + shop/finance
Communication consent	Lawful communication/preferences	User + communications system

A public member directory should be **opt-in**, not automatically generated from the CRM.

Pastoral notes deserve additional safeguards:

- no global admin read by default,
- explicit owner/access list,
- audit each access,
- do not put unnecessary detail in ordinary CRM notes,
- consider application-level encryption for particularly sensitive free-form notes,
- defined retention policy.

GDPR and POPIA

Do not claim the Jamaican church is automatically subject to either regulation merely because its website is globally reachable.

GDPR can apply outside the EU where an organization offers goods/services to people in the EU or monitors their behaviour, among other territorial-scope conditions. ³⁶ POPIA compliance should likewise be assessed if South African jurisdictional/processing conditions become relevant; South Africa's Information Regulator provides obligations around security compromises and Information Officers where POPIA applies. ³⁷

The pragmatic solution is to build privacy capabilities that make compliance easier regardless:

- consent records,
- export/access workflow,
- correction,

- erasure where legally permitted,
- retention automation,
- data portability,
- marketing opt-out,
- breach response,
- processor/vendor inventory.

Backups require special attention.

Supabase Pro includes daily database backups with seven-day retention. ⁶ Point-in-Time Recovery is available as an additional capability on supported plans/configurations. ³⁸

However, a critical detail is that **Supabase database backups do not back up objects stored through the Storage API.** ³⁹

Therefore use:

Supabase PostgreSQL

- └─ daily platform backup
- └─ optional PITR
- └─ scheduled logical export

Supabase Storage

- └─ external object copy
- └─ versioned backup destination
- └─ manifest/checksum

Quarterly

- └─ full restore rehearsal

Supabase supports logical database backups through its CLI/ `pg_dump` tooling. ⁴⁰

A backup is not proven until a restore has succeeded.

Hosting comparison

Platform	Fit	Advantages	Trade-offs	Recommendation
Vercel + Supabase	Excellent	Best direct Next.js workflow, preview deployments, CDN, managed deployment	Usage-based scaling requires monitoring	Recommended default

Platform	Fit	Advantages	Trade-offs	Recommendation
Cloudflare + Supabase	Very good	Strong edge/CDN/ security ecosystem; good fit when Stream is used	Next.js runtime compatibility should be tested with chosen adapter	Good optimization path
Docker/ Node host + Supabase	Good	Portability and infrastructure control	More DevOps responsibility	Useful where organizational hosting requirements demand it

Next.js officially supports deployment as a Node.js server, Docker container, static export or through platform adapters. ⁴¹ Vercel Pro currently starts at US\$20/month and includes automatic CI/CD, CDN, WAF and DDoS mitigation features. ⁷

Recommended CI/CD



CI should run:

```

TypeScript
ESLint
unit tests
RLS/database tests
API tests
build
accessibility smoke tests
Playwright end-to-end tests
dependency scan
secret scan
  
```

Database migrations should be immutable, versioned SQL committed with the application. Do not make production schema changes manually and then “fix Git later.”

Current planning cost model

These are engineering planning estimates rather than vendor quotations. Payment transaction fees, SMS, high video usage, taxes and custom enterprise support are excluded.

Supabase Pro currently starts at US\$25/month, while Team starts at US\$599/month. Pro includes 100,000 MAU and paid plans receive US\$10 monthly compute credit; current compute examples include Micro US\$10, Medium US\$60 and Large US\$110 per month before applicable credits. ⁶

Vercel Pro starts at US\$20/month. ⁷ SendGrid Essentials starts around US\$19.95/month. ⁴² Cloudflare Stream's published pricing is based on stored and delivered video minutes, currently US\$5 per 1,000 stored minutes and US\$1 per 1,000 delivered minutes. ⁴³

Scale	Example workload	Approx. platform budget / month
Small church launch	<5k member/visitor accounts, moderate web traffic, public YouTube, light email	US\$50–120 + payment/SMS/domain
Medium platform	5k–30k accounts, regular campaigns, owned video, shop, automation	US\$150–500 + payment/SMS/video overages
Large / multi-site	High traffic, many campuses, significant streaming, larger DB, advanced backup/support	US\$700–3,000+
Enterprise network	Many churches/campuses, dedicated support/SLA/security requirements	Custom; potentially several thousand dollars/month+

The jump at the large end is partly because Supabase Team begins at US\$599/month. ⁶ It should not be adopted simply because it is “enterprise-looking”; stay on Pro until its Team features and operational support are actually needed.

Twilio needs its own variable budget. At the currently listed Jamaica outbound rate around US\$0.3151/segment, sending a single-segment SMS to 1,000 recipients would be roughly **US\$315** before any additional applicable charges, which is why SMS should not be the default bulk newsletter channel. ²⁹

Experience design, accessibility, SEO, and content strategy

The phrase “best church website in Jamaica” should not mean the most animations or the most technology. It should mean:

fastest path from a person's question to their next meaningful step with the church.

For a first-time visitor landing on the home page, five questions should be answered immediately:

Who are you?
Where are you?
When do you meet?
What can I expect?
What can I do next?

The highest-value home-page actions should be:

Plan Your Visit
Watch Live / Latest Sermon

Upcoming Events
Get Connected
Give
Shop

For the Bull Bay site, I would build the visual identity from the church's real logo, church photography and Jamaican/Bull Bay context rather than generic American church stock photography.

Recommended home-page structure

```
Navigation
|
|— Hero
|   |— Church identity
|   |— Next service
|   |— Plan Your Visit / Watch / Give
|
|— Welcome message
|
|— This week's service
|
|— Upcoming events
|
|— Latest sermon
|
|— Ministries / Find your place
|
|— Small groups
|
|— Community impact / outreach
|
|— Giving story
|
|— Church shop
|
|— Location + directions
|
|— Footer / contact / social
```

Mobile first must be a requirement, not a later responsive pass.

Every primary function should work comfortably from a phone:

- event registration,
- giving,

- cart/checkout,
- sermon playback,
- prayer request,
- volunteer confirmation,
- member profile update,
- check-in,
- admin approval.

Avoid heavy autoplay hero video. Reserve network bandwidth for content the user deliberately requests.

Accessibility should target **WCAG 2.2 AA**. WCAG 2.2 is a current W3C Recommendation built around perceivable, operable, understandable and robust content, with A/AA/AAA conformance levels. ⁴⁴

The acceptance criteria should include:

Area	Requirement
Keyboard	Every control operable without mouse
Focus	Highly visible focus state
Headings	Logical document hierarchy
Forms	Explicit labels, errors and instructions
Images	Meaningful alternative text
Sermons	Captions and preferably transcript
Audio	Transcript when practical
Color	Sufficient contrast; information never conveyed by color alone
Motion	Respect reduced-motion preference
Dialogs	Focus trapping/return handled correctly
Navigation	Skip link and landmarks
Validation	Screen-reader accessible errors
Zoom	Layout remains usable at large zoom
Mobile	Large, separated interaction targets

Captions and transcripts are not merely an accessibility add-on: they also make sermons searchable and reusable as study material.

SEO architecture

Google recommends creating useful, people-first content and making sites technically accessible to search engines. ⁴⁵

Every indexable page needs:

- unique `<title>`,
- meta description,
- canonical URL,
- Open Graph metadata,
- clean human-readable slug,
- internal linking,
- sitemap membership,
- appropriate structured data.

Use:

Page	Schema.org/Google structured data
Church/home	<code>Organization</code>
Event	<code>Event</code>
Sermon video	<code>VideoObject</code>
Product	<code>Product</code> + <code>Offer</code>
Articles/news	<code>Article</code> where appropriate

Google publishes dedicated guidance for Organization, Event, VideoObject and Product structured data.

46

A sermon page might expose:

```
{
  "@context": "https://schema.org",
  "@type": "VideoObject",
  "name": "Built on the Rock",
  "description": "Sermon summary...",
  "thumbnailUrl": [...],
  "uploadDate": "2026-08-16T...",
  "duration": "PT42M10S"
}
```

The site should also synchronize local information with the church's Google Business Profile so name, address, phone, service information and directions do not contradict each other.

Performance should be measured with Core Web Vitals rather than subjective statements that the site “feels fast.” Google's current Core Web Vitals focus on loading, interaction responsiveness and visual stability through LCP, INP and CLS. 47

Recommended sitemap

```
/
├── about
│   ├── our-story
│   ├── beliefs
│   ├── leadership
│   └── locations
│       └── bull-bay
├── visit
│   ├── what-to-expect
│   ├── service-times
│   └── directions
├── events
│   └── [event-slug]
├── calendar
├── sermons
│   ├── [sermon-slug]
│   ├── series
│   │   └── [series-slug]
│   ├── speakers
│   └── topics
├── live
├── ministries
│   ├── children
│   ├── youth
│   ├── men
│   ├── women
│   ├── families
│   └── outreach
├── groups
│   └── [group-slug]
├── serve
│   └── [opportunity-slug]
├── prayer
└── contact
```

```
|
|— give
|
|— shop
|   |— [product-slug]
|   |— cart
|   └─ checkout
|
|— news
|   └─ [post-slug]
|
|— member
|   |— profile
|   |— household
|   |— events
|   |— groups
|   |— serving
|   |— giving
|   |— orders
|   |— downloads
|   └─ notifications
|
|— pastor
|   |— dashboard
|   |— sermons
|   |— care
|   |— services
|   └─ communications
|
└─ admin
    |— dashboard
    |— people
    |— visitors
    |— events
    |— check-in
    |— groups
    |— volunteers
    |— content
    |— media
    |— giving
    |— shop
    |— communications
    |— reports
    |— campuses
    └─ roles
```

- └─ audit
- └─ integrations

The content strategy should follow a predictable weekly rhythm:

Frequency	Content
Every service	Sermon video/audio + title + Scripture + summary
Weekly	Events/current announcement refresh
Weekly	Home-page next-service details
Monthly	Community/outreach story
Monthly	Ministry spotlight/testimony
Quarterly	About/leadership/service information audit
Continuous	Shop inventory/products
Continuous	Group/volunteer availability

The church's strongest long-term SEO asset will likely be the accumulating sermon, ministry, event and community-content library rather than generic static landing pages.

Delivery package, MVP, roadmap, and next-step checklist

The downloadable project package is:

[Download](#) `bull-bay-church-starter.zip`

Its current structure is:

```
bull-bay-church-starter/  
└─ app/  
  └─ api/  
    └─ giving/checkout/  
    └─ shop/checkout/  
    └─ webhooks/paypal/  
  └─ admin/  
  └─ cart/  
  └─ events/  
  └─ give/  
  └─ member/  
  └─ sermons/  
  └─ shop/  
  └─ globals.css
```

```

├── layout.tsx
├── page.tsx
├── components/
│   └── AddToCartButton.tsx
├── lib/
│   ├── demo-data.ts
│   └── supabase/
│       ├── client.ts
│       └── server.ts
├── supabase/
│   ├── migrations/
│   │   └── 0001_init.sql
│   └── seed.sql
├── docs/
│   ├── ARCHITECTURE.md
│   ├── DATA_MODEL.md
│   └── SECURITY.md
├── scripts/
│   └── deploy.sh
├── .github/
│   └── workflows/
│       └── ci.yml
├── .env.example
├── next.config.ts
├── package.json
├── README.md
├── tsconfig.json
└── vercel.json

```

The starter follows Supabase's current SSR model using `@supabase/ssr`, which Supabase recommends for server-rendered JavaScript frameworks such as Next.js. ⁴⁸

It intentionally does **not** accept real payments yet. The included checkout endpoint documents the production sequence instead:

```

// Production checkout:
//
// 1. Do not trust prices supplied by the browser.

```

```
// 2. Re-read product/variant prices from PostgreSQL.
// 3. Validate stock.
// 4. Create pending order transactionally.
// 5. Reserve inventory.
// 6. Create gateway checkout.
// 7. Redirect user to payment provider.
// 8. Verify signed provider webhook.
// 9. Mark order paid.
// 10. Create digital entitlements if applicable.
```

This is a feature, not an omission: live payment code should only be written after the church chooses and opens the relevant merchant account.

Recommended MVP

The MVP should resist the temptation to build every church-management feature simultaneously. The best first release is:

MVP component	Included
Public home/about/visit	✓
Campuses/location foundation	✓
Events + registrations	✓
Public calendar + ICS	✓
Sermon library	✓
Livestream page	✓
Contact/visitor form	✓
Prayer-request form	✓
Giving through one approved gateway	✓
Shop catalog	✓
Physical + digital products	✓
Cart/checkout/orders	✓
Digital entitlement delivery	✓
Member authentication	✓
Member profile	✓
Giving history	✓

MVP component	Included
Order/download history	✓
Admin people CRM	✓
Admin events	✓
Admin sermons/media	✓
Admin giving	✓
Admin shop/orders	✓
Roles/RLS/MFA/audit	✓
Basic reporting	✓
Full child check-in	Phase after MVP
Advanced group attendance	Phase after MVP
Volunteer scheduling engine	Phase after MVP
Full pastoral-care case system	Phase after MVP, security review first
Semantic AI search	Later
Full accounting API sync	Later
Advanced workflow builder	Later
Native iOS/Android app	Later

This still creates a substantial product, but every MVP component contributes directly to public ministry, revenue/giving, content or core administration.

A realistic implementation sequence is:

Phase	Main deliverable
Foundation	Brand system, information architecture, Supabase project, schema, auth, RLS, CI/CD
Public experience	Home, visit, ministries, events, calendar, sermons, livestream, contact
Identity	Login, profiles, households, preferences, staff invitations, MFA
Giving	Funds, donations, gateway, webhooks, reconciliation, receipts
Commerce	Products, cart, checkout, inventory, orders, digital delivery, refunds
Church operations	People, groups, volunteers, event registrations/attendance

Phase	Main deliverable
Pastoral workspace	Sermon planning, care permissions/workflows, service planning
Communications	SendGrid, SMS, notifications, segments
Insights	Reporting, GA4, operational dashboards
Scale	Multi-campus, accounting, automation, semantic search
Hardening	Accessibility audit, penetration testing, privacy review, disaster-recovery test

Definition of done for launch should be stronger than “the pages work.”

A launch should not happen until all of the following are true:

- ☐ Church legal/entity name and merchant details are confirmed.
- ☐ Exact service times, leadership, ministry descriptions and location information are approved by church staff.
- ☐ Supabase production project is separated from development.
- ☐ RLS is enabled and tested for every browser-accessible table.
- ☐ Anonymous users cannot retrieve member/private records.
- ☐ Members cannot retrieve another member's giving/order history.
- ☐ Pastoral records have explicit care-team authorization.
- ☐ Ordinary website admins cannot see pastoral notes.
- ☐ Ordinary content editors cannot see donations.
- ☐ Staff MFA is enforced.
- ☐ Payment gateway contract is approved for the Jamaican legal entity.
- ☐ Provider webhooks are cryptographically verified.
- ☐ Duplicate payment webhooks cannot create duplicate fulfilment.
- ☐ Refund authorization is tested.
- ☐ Digital product URLs are private and expire.
- ☐ Physical inventory reservations and refund/restock flows are tested.
- ☐ Jamaican tax/accounting treatment is reviewed professionally.
- ☐ Data-controller registration/compliance obligations with Jamaica's OIC are addressed.
- ☐ Privacy notice and communication consents are approved.
- ☐ Data retention/deletion schedules are documented.
- ☐ Breach-response plan includes Jamaica's 72-hour OIC requirement. ³⁴
- ☐ Database backup is configured.
- ☐ Storage-object backup is configured separately because Supabase database backups do not include Storage objects. ³⁹
- ☐ At least one restore test has succeeded.
- ☐ WCAG 2.2 AA audit is completed. ⁴⁹
- ☐ Sermon captions/transcripts are part of the publishing workflow.
- ☐ Structured data is implemented for church, events, video and products. ⁴⁶
- ☐ Mobile checkout, event registration and giving are tested on real phones.
- ☐ Core Web Vitals are measured.
- ☐ Analytics excludes unnecessary sensitive/member/pastoral information.

- [] Production secrets exist only server-side.
- [] Security headers/CSP are tested.
- [] Dependency and secret scanning run in CI.
- [] An audit trail exists for permissions, sensitive exports and refunds.
- [] Error monitoring and uptime monitoring are configured.
- [] Google Business Profile details and website details are consistent.
- [] A documented content owner exists for events, sermons, shop, ministries and home-page information.

The most important immediate decisions before turning the starter into a production application are therefore:

First, confirm the church's legal organization, merchant-bank setup and whether PayPal, WiPay, Powertranz or a combination offers the best Jamaican settlement/currency/fee arrangement. PayPal already offers Jamaica-specific checkout and recurring-payment functionality; WiPay and Powertranz warrant parallel commercial evaluation because of their Caribbean focus. ⁵⁰

Second, establish the church's data-governance model before importing member information. Jamaica's OIC requires registered data controllers to meet substantive security, minimization, retention and breach-management obligations. ⁵¹

Third, implement identity and authorization before building administrative screens. The correct order is:

```
Auth
→ organization/campus
→ profile
→ roles
→ permissions
→ RLS
→ audit logging
→ admin features
```

not the reverse.

Fourth, treat giving, commerce and pastoral care as separate security domains even though they share the same Supabase project.

Fifth, make the church's content engine—events, sermons, livestreams, ministries and community stories—as strategically important as the management system. A technically sophisticated backend paired with an outdated public website would not satisfy the objective of becoming Jamaica's strongest church digital presence.

With that architecture, the Bull Bay church platform can begin as a fast, polished single-church website and evolve without a rebuild into a multi-campus church operating system, media platform, e-commerce business, giving platform and member community—while keeping the church's canonical data under its own Supabase-backed application rather than fragmented across unrelated systems.

1 The latest Next.js news

https://nextjs.org/blog?utm_source=chatgpt.com

2 9 Auth | Supabase Docs

https://supabase.com/docs/guides/auth?utm_source=chatgpt.com

3 8 Planning Center People: Membership Management

https://get.planningcenteronline.com/people?utm_source=chatgpt.com

4 18 Stripe global availability

https://stripe.com/global?utm_source=chatgpt.com

5 15 Payment Solutions for Your Business | PayPal JM

https://www.paypal.com/jm/webapps/mpp/merchant?utm_source=chatgpt.com

6 Pricing & Fees

https://supabase.com/pricing?utm_source=chatgpt.com

7 Vercel Pricing: Hobby, Pro, and Enterprise plans

https://vercel.com/pricing?utm_source=chatgpt.com

10 48 Creating a Supabase client for SSR

https://supabase.com/docs/guides/auth/server-side/creating-a-client?utm_source=chatgpt.com

11 Row Level Security | Supabase Docs

https://supabase.com/docs/guides/database/postgres/row-level-security?utm_source=chatgpt.com

12 Storage Access Control | Supabase Docs

https://supabase.com/docs/guides/storage/security/access-control?utm_source=chatgpt.com

13 Storage Buckets | Supabase Docs

https://supabase.com/docs/guides/storage/buckets/fundamentals?utm_source=chatgpt.com

14 Multi-Factor Authentication | Supabase Docs

https://supabase.com/docs/guides/auth/auth-mfa?utm_source=chatgpt.com

16 WiPay Payments — Card, QR, Links, POS

https://wipaycaribbean.com/?utm_source=chatgpt.com

17 Powertranz

https://www.firstatlanticcommerce.com/?utm_source=chatgpt.com

19 Create recipients - Stripe Documentation

https://docs.stripe.com/global-payouts/recipient-creation-options?locale=en-GB&utm_source=chatgpt.com

20 Merchant Resources

https://www.pcisecuritystandards.org/merchants/?utm_source=chatgpt.com

21 If a merchant's e-commerce implementation meets the ...

https://www.pcisecuritystandards.org/faqs/if-a-merchant-s-e-commerce-implementation-meets-the-criteria-that-all-elements-of-payment-pages-originate-from-a-pci-dss-compliant-service-provider-is-the-merchant-eligible-to-complete-saq-a-or-saq-a-ep/?utm_source=chatgpt.com

22 Gov't Projects \$29.4B From New Revenue Measures In ...

https://jis.gov.jm/govt-projects-29-4b-from-new-revenue-measures-in-fiscal-year-2026-27/?utm_source=chatgpt.com

23 Overview · Cloudflare Stream docs

https://developers.cloudflare.com/stream/?utm_source=chatgpt.com

24 Bind to Workers API · Cloudflare Stream docs

https://developers.cloudflare.com/stream/manage-video-library/bindings/?utm_source=chatgpt.com

25 Captions | YouTube Data API

https://developers.google.com/youtube/v3/docs/captions?utm_source=chatgpt.com

26 Semantic search | Supabase Docs

https://supabase.com/docs/guides/ai/semantic-search?utm_source=chatgpt.com

27 RAG with Permissions | Supabase Docs

https://supabase.com/docs/guides/ai/rag-with-permissions?utm_source=chatgpt.com

28 42 Twilio SendGrid Email API pricing

https://sendgrid.com/en-us/solutions/add-ons/additional-dedicated-ip-address?utm_source=chatgpt.com

29 SMS Pricing in Jamaica for Text Messaging

https://www.twilio.com/en-us/sms/pricing/jm?utm_source=chatgpt.com

30 Google Calendar API overview

https://developers.google.com/workspace/calendar/api/guides/overview?utm_source=chatgpt.com

31 Authorization

https://developer.intuit.com/app/developer/qbo/docs/develop/sdks-and-samples-collections/php/authorization?utm_source=chatgpt.com

32 REST hook triggers: Zapier CLI webhook subscription guide

https://docs.zapier.com/integrations/build/cli-hook-trigger?utm_source=chatgpt.com

33 OWASP Application Security Verification Standard (ASVS)

https://owasp.org/www-project-application-security-verification-standard/?utm_source=chatgpt.com

34 35 51 Obligations of Data Controllers under the Data Protection ...

https://oic.gov.jm/press-release/obligations-data-controllers-under-data-protection-act-dpa?utm_source=chatgpt.com

36 REGULATION (EU) 2016 - EUR-Lex - European Union

https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX%3A32016R0679&utm_source=chatgpt.com

37 FACT SHEET: HANDLING OF SECURITY COMPROMISES -

https://infoeregulator.org.za/2025/08/19/fact-sheet-handling-of-security-compromises/?utm_source=chatgpt.com

38 Database Backups | Supabase Docs

https://supabase.com/docs/guides/platform/backups?utm_source=chatgpt.com

39 Database | Supabase Docs

https://supabase.com/docs/guides/database/overview?utm_source=chatgpt.com

40 Troubleshooting | How to download logical backups in ...

https://supabase.com/docs/guides/troubleshooting/download-logical-backups?utm_source=chatgpt.com

41 Getting Started: Deploying

https://nextjs.org/docs/app/getting-started/deploying?utm_source=chatgpt.com

43 Pricing

https://www.cloudflare.com/plans/?utm_source=chatgpt.com

44 49 **What's New in WCAG 2.2 | Web Accessibility Initiative (WAI)**

https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/?utm_source=chatgpt.com

45 **Search Engine Optimization (SEO) Starter Guide**

https://developers.google.com/search/docs/fundamentals/seo-starter-guide?utm_source=chatgpt.com

46 **Organization Schema Markup | Google Search Central**

https://developers.google.com/search/docs/appearance/structured-data/organization?utm_source=chatgpt.com

47 **Core Web Vitals**

https://web.dev/explore/learn-core-web-vitals?utm_source=chatgpt.com

50 **Accept Online Payments with PayPal | PayPal JM**

https://www.paypal.com/jm/business/accept-payments/checkout?utm_source=chatgpt.com